Chapter route

How does layered Windows architecture organize objects, processes, I/O, and services across user/kernel boundaries?

Prerequisites are OS structure, protection domains, and file/I/O mechanisms.

- Identify user/kernel-mode components.
- Analyze object-manager handles and granted access.
- Separate Windows 7 history from contemporary implementations.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One NT-family case

The case distinguishes user subsystems/services from kernel-mode executive, kernel, HAL, drivers. It does not substitute DOS-based Windows, Mach, or later Windows additions.

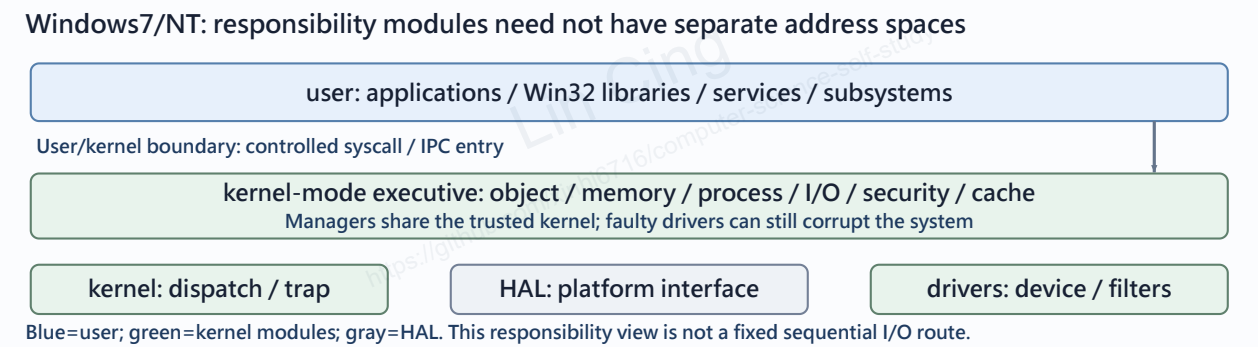
Term	Meaning	Does not imply
Win32 API	Application interfaces for processes, objects, I/O	Every API makes a syscall
NT kernel/executive	Trusted scheduling, memory, object, I/O	Every component has its own address space
Compatibility	Runs existing programs in specified environment	Every ISA, bitness, legacy subsystem supported
Portability	Concentrates hardware dependencies	Driver binaries run unchanged across CPUs

OS/2, Win16, POSIX labels in historical diagrams retain their period/version scope. They do not make every Windows 7 installation run every old binary. Subsequent cases specify execution, configuration, failure assumptions.

ReadFile across trusted components

Library preparation enters a controlled kernel interface that checks handle/buffer. I/O manager, filesystem, cache, memory manager select a hit or driver path. Device waits can block the thread before completion returns a result.

An API name alone specifies neither blocking, copy count, IRP presence, nor context-switch count. Cached/fast paths differ. Record open flags, object type, cache state, and completion mode first. Layering includes many kernel-mode components rather than a mandatory user-server hop or independent failure domain per component.



Goals and mechanisms

Goal	Mechanism	Not guaranteed by its name
Security	Tokens, descriptors, granted handle access	Bug-free kernel/drivers
Reliability	Boundaries, references, error recovery	Shared-kernel fault containment
Performance	Caches, async I/O, scheduling	Uniformly low latency
Extensibility	Object/driver interfaces	Every extension is safe
Portability	HAL, architecture-specific layers	Cost-free porting
Internationalization	Unicode, locale data	Characters equal bytes
Power/dynamic devices	Power states, PnP lifetime	Removal without outstanding I/O

HAL abstracts selected platform, interrupt, timer details without removing every memory-model/driver responsibility. SMP support does not make shared updates race-free.

Unicode length units

Win32 W interfaces use UTF-16 code units; A interfaces use code-page conversion. Neither count is necessarily a UTF-8 byte count or a Unicode scalar count.

A, U+1F600, B encodes as 0041 D83D DE00 0042: four 16-bit code units, three scalars, and 8 data bytes. If a terminating NUL is required, it adds 2 bytes, for 10 total; a 4-byte buffer cannot hold the complete data and terminator.

A capacity in wchar_t units is not a byte count, and a byte length is not a scalar count. Unicode APIs do not automatically normalize every filename into one canonical form.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

PnP/power depend on ownership

The lifecycle model is working→stop-pending→stopped→removed, not the complete PnP IRP list. Stop-pending blocks new submission, drains/cancels existing work with confirmed completion, then releases device context, DMA maps, buffers.

Freeing context before an outstanding IRQ/request ends causes use-after-free. Requested cancellation does not prove DMA stopped. Removal can finish requests with errors rather than guaranteed success. No resource can pass to an incompatible owner while execution/device references can still touch it.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Kernel, executive, HAL, and services

Component	Responsibility	State
Kernel dispatcher	Select/dispatch threads, synchronization/trap foundations	Ready queues, thread states
Executive managers	Objects, memory, process, I/O, security, cache	Handles, PTE/PFN, IRPs
HAL	Platform, interrupt, timer abstraction	Routing/control
Drivers	Devices, filters, filesystems	Device objects, queues, buffers
User services/subsystems	Login, background services, application environments	User address spaces, IPC

Kernel/executive are responsibility labels, both privileged. A thread entering kernel uses its kernel stack without creating a separately scheduled second thread.

Thread priority differs from IRQL

Dispatcher chooses the highest-priority eligible ready thread:variable 1–15, realtime 16–31, idle special 0. Variable boosts decay no lower than base and do not enter realtime range. Conditions affect boost/quantum;the later q2 is not universal.

IRQL governs current-processor interrupt/kernel activity and permitted operations, not another 0–31 thread priority.

Common x86/x64 IRQL	Execution	Constraint
PASSIVE_LEVEL0	Threads/workers	Waiting still follows API rules
APC_LEVEL1	Selected kernel APC work	Specific APC/call constraints
DISPATCH_LEVEL2	Ordinary DPC/spinlock regions	No nonzero blocking wait/pageable access fault
Device DIRQL	ISR/device synchronization	Shorter, more restricted work

Raising one CPU's IRQL does not exclude other CPUs. Realtime thread priority does not guarantee hard deadlines. Driver priority rules define these constraints.

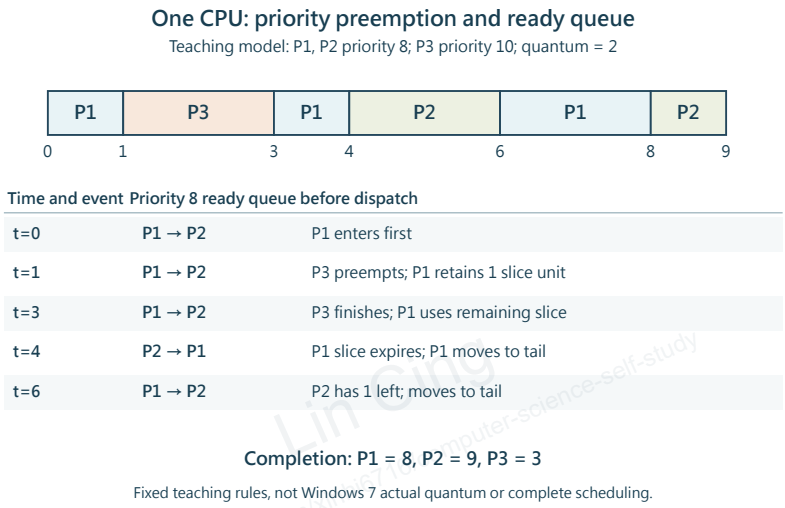
A fixed priority trace

One CPU, no boosts/decay, equal-priority q2, higher priority immediately preempts. A preempted task keeps residual slice and returns to its priority queue's front. P1/P2 priority 8, ready at 0, bursts 4/3. P3 priority 10, ready at 1, burst 2.

Interval	Reason	Result
0–1 P1	First equal-priority arrival	Preempted with burst 3/slice 1
1–3 P3	Higher priority 10	Complete
3–4 P1	Queue front, residual slice	Burst2, quantum expires, tail
4–6 P2	Priority8 queue front	Burst1, tail
6–8 P1	Queue front	Complete
8–9 P2	Only ready task	Complete

Completions P1/P2/P3=8/9/3, total CPU 9. This isolates queue/quantum rules; actual boosts, quanta, processors require their own calculation.

Priority preemption and queue trace



P3 preempts P1 at t=1; P1 retains one unit of its slice. P1 resumes at t=3 and moves to the priority 8 queue tail when its slice expires at t=4. Completion times are P1=8, P2=9, and P3=3. These are teaching rules, not a Windows 7 scheduling guarantee.

Dispatcher objects are not universal queues

Manual-reset events remain signaled until ResetEvent and can release many waiters. Auto-reset satisfies one wait then resets. Repeated SetEvent on an already signaled event does not accumulate independent tokens. Counted resources need semaphores or explicit queues.

A mutex has an owner. WAIT_ABANDONED signals abnormal prior-owner termination and requires checking protected-data consistency;acquisition does not repair it. Mutexes alone do not prevent multi-lock ordering deadlocks.

This model exposes semaphore count 0…limit. At 0, waiters sleep; release can hand off a permit. Do not confuse it with Chapter 5's alternative internal signed counter.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

ISR, DPC, APC, and worker contracts

ISR checks/acknowledges source and performs minimum work. Ordinary DPC defers work but remains DISPATCH_LEVEL, so it cannot perform arbitrary blocking filesystem I/O. Sleepable/pageable work goes to an appropriate PASSIVE worker with retained context references.

An APC targets a thread. Ordinary user APCs generally run during its alertable wait, not arbitrary user instructions. Sleep differs from SleepEx(TRUE). Special kernel APC/startup paths have additional rules outside this summary.

ISR→DPC→worker retains context before queueing and releases after worker completion. Freeing before the queued worker uses it is invalid. IRQL and ownership jointly determine legal work.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Handles are process-table indexes

A's 0x40→F, granted R while B's 0x40 can name event E. Integers alone do not transfer authority. Inheritance, DuplicateHandle, or named reopening provide explicit authorized sharing.

Step	A table	B table	F lifetime
Initial	0x40→F, R	Empty	A retains
SAME_ACCESS duplicate	Unchanged	0x88→F, R	Both retain
A closes	Empty	Still 0x88	Retained
B closes, internal I/O remains	Empty	Empty	Still retained
Last internal reference ends	Empty	Empty	Cleanup permitted by type rules

SAME_ACCESS preserves rights. Other desired-access behavior depends on object/API authority, not a universal subset rule. Source/target process handles need rights such as PROCESS_DUP_HANDLE. DuplicateHandle.

Tokens and ordered DACLs

Choose the effective token, including impersonation, with SIDs, groups, privileges. Descriptor contains owner, DACL, SACL;DACL grants/denies while SACL principally audits.

This simplified model uses enabled SIDs, ordinary ACEs, specific rights, excluding privilege, integrity, owner/object-ACE exceptions. Abstract R=1/W=2 are not actual Win32 masks. Request R/W starts remaining 3.

ACE	Match	Remaining/result
ALLOW Alice R	Yes	RemoveR, leaveW
DENY Staff W	Alice inStaff	IntersectsW, deny/stop
ALLOW Staff RW	Not reached	Cannot repair denial

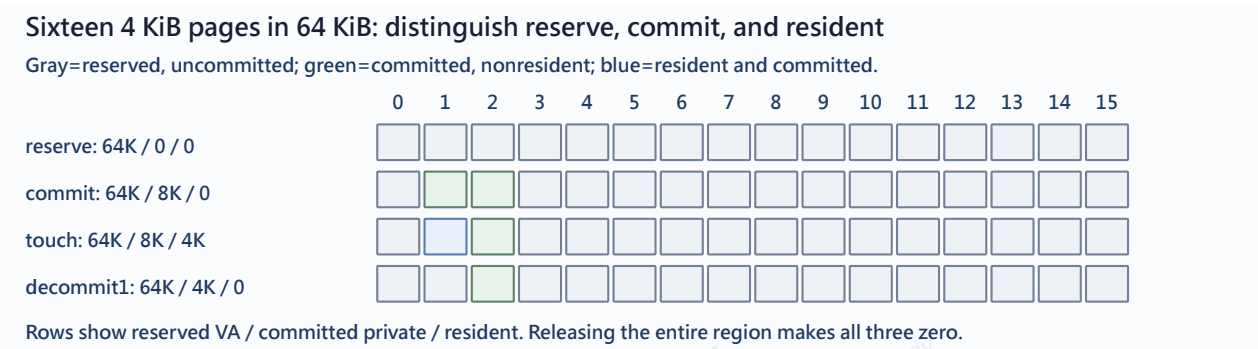
Moving the final allow before deny can satisfy all remaining bits first. Order matters;canonical ACE ordering is preferred. NULL DACL imposes no DACL restriction;empty DACL grants nothing in this ordinary model. Ordered access checks.

Reservation, commitment, and residency

Fix 4 KiB pages, 64 KiB allocation granularity, private base 0x100000. Reserve 64 KiB of process VA, then commit pages 0x101000/0x102000 with 8 KiB charge, not necessarily physical frames.

Step	Reserved VA	Private committed	Resident here
Reserve	64 KiB	0	0
Commit two pages	64 KiB	8 KiB	0
Touch 0x101123	64 KiB	8 KiB	4 KiB zero-fill
Decommit first, no other reference	64 KiB	4 KiB	0
Release region	0	0	0

Reservation, commitment, and residency (continued)



Reserve/commit can fail;commit follows system backing commitments rather than immediately zero-writing pagefile pages. Equal VA numbers in different processes do not compete for one shared virtual address. VirtualAlloc.

PTE, PFN, and working-set trimming

PTEs describe virtual mappings/software states;PFN database describes frames, references, lists. A legal nonresident page can be demand-zero, section/file/pagefile-backed, or recoverable from RAM. Not every fault reads storage.

VA 0x401123 has VPN 0x401, offset 0x123. A valid PTE→PFN 0x2A gives PA 0x2A123. Writing RO without legal COW is an access violation, not permission to allocate around protection.

Trimmed clean backed pages can enter standby rather than vanish. Modified pages need writeback before reuse;age alone cannot free pinned/I/O-held frames. Standby recovery can be a soft fault;missing RAM backing may require storage. Windows 7 reference-history/working-set heuristics are neither future-aware OPT nor exact LRU. Track frames, PTEs, pending writes together.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Processes, sections, and ALPC authority

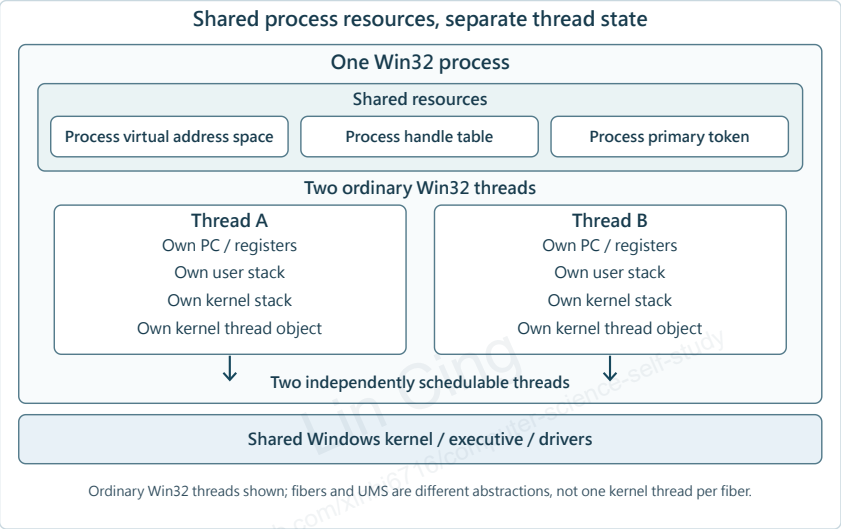
CreateProcess includes an initial thread;another CreateThread makes two. Process owns address space, handles, token;threads own registers, user/kernel stacks, execution state. Closing a process handle does not terminate it. Exited-thread objects can remain observable/waitable through retained handles.

Sections map file/pagefile backing at different process VAs with independent rights and shared-data synchronization. ALPC/ports package client/server requests and authority. Shared sections save copies without removing lengths/ownership checks.

A maps offset 0 at 0x100000, B at 0x500000. A+16 and B+16 can share bytes, but literal pointer 0x100010 is not B's pointer. Transfer section-relative offset 16 and length.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Two Win32 threads in one process



Each ordinary Win32 thread has a kernel-schedulable thread object and its own execution state. Threads share the process address space and handle table, but have separate registers, user stacks, and kernel stacks. The token shown is the process primary token; thread impersonation changes which identity is effective.

Fibers and UMS are other user-level scheduling abstractions; a fiber is not automatically a kernel thread. Shared resources still require synchronization.

IRP dispatch down, completion up

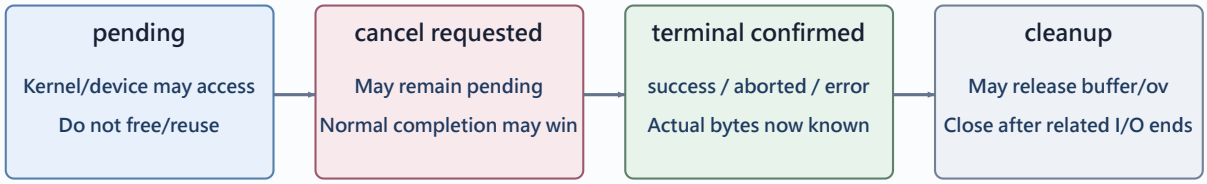
I/O manager can send IRPs through filesystem/filter/device stacks with per-stack state. Lower completion returns status/actual bytes through completion paths. Cached/fast operations need not repeat the same full chain.

Time	Ownership/state	Allowed action
Submit	Valid buffer, unique operation state	Do not reuse while pending
Pending	Driver/device can access	Cancel requests work;does not free
Hardware/cancel completion	Status/bytes established	Notify under completion rules
Caller confirms terminal	Success/aborted/error known	Release buffer/OVERLAPPED

CancelIoEx races normal completion. Success means cancellation requested;ERROR_NOT_FOUND can mean no matching pending operation. Collect the original result regardless. Cancellation contract.

IRP dispatch down, completion up (continued)

Cancel requested ≠ I/O complete: retain buffer/OVERLAPPED until terminal



This path requests cancellation. Pending may complete normally; cancel success is not the operation result.
Use unique ov/buffer per outstanding operation. Win7 needs a count argument; final bytes come from completion.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Cache/memory managers and reads

Cached reads find file ranges in filesystem/cache views. Resident data copies to caller. A view fault invokes paging read through storage to fill a frame, then satisfies the original read. Paging/cached requests are related but not two application deliveries.

For 4096 B, a hit copies 4096 to caller. A miss adds 4096 B device→cache DMA before that copy. CPU copy cost, DMA bytes, storage reads differ; this is not two CPU copies. Lazy writing batches dirty data while delaying durability. Filesystem/storage log/data flush ordering establishes persistence rather than hit/write success alone.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

PnP, power, registry, and boot dependencies

PnP coordinates discovery, driver stacks, removal;power manager coordinates power states. Drain hardware-touching requests before releasing resources. Driver objects, device objects, and physical presence/usability have different lifetimes.

Registry hives, keys, values store configuration, possibly volatile state, rather than ordinary file paths. Changes take effect under service reload/restart/notification rules, not instantaneously in every process.

Boot progresses through firmware/boot manager/OS loader, kernel/HAL/boot drivers, kernel initialization, sessions/services, logon. BIOS/UEFI differ;no universal MBR file sequence is assumed. Page tables/storage drivers must exist before dependent memory/file services.

Session, window station, desktop, and process

Terminal services/fast user switching preserve interactive sessions, desktop/input views, and processes. Windows 7 service/session arrangements must not be confused with older versions.

Resource	Independence across sessions	Reason
User address space	Per-process mappings	Session numbers do not replace MMU
Window stations/desktops, some names	Session-related scope	Object rights/namespace still matter
Kernel, drivers, RAM	Shared	Common faults/resource limits
NTFS/network services	Can be shared	Credentials/object policy

The abstraction does not prove unrestricted simultaneous interactive connections;product, licensing, deployment limits are separate.

Local/Global scope is not authorization

S1/S2 Local\Ready events can be distinct session objects. Authorized Global\Ready access can reach one global object while retaining DACL/create/open rights. Global never means bypass checks.

Handle 0x80 in different processes can denote unrelated objects. Equal names require matching namespace and authority. Session ID is not trusted user identity. Switching the foreground user need not terminate background-session processes or release their locks, CPU, memory.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Volumes, MFT, attributes, extents

Windows 7 supports multiple filesystems;not every volume is NTFS. Clusters allocate storage;LCNs are volume-relative. Disk offsets additionally include partition/volume start.

MFT records contain attributes. Small data can be resident in its record;nonresident data uses VCN→LCN extents. Nonresident here does not mean absent from RAM. In a 1024 B record with 700 already used,324 remains. Data 300 fits this budget;512 does not. Names/attributes/format change the threshold;300 is not universal.

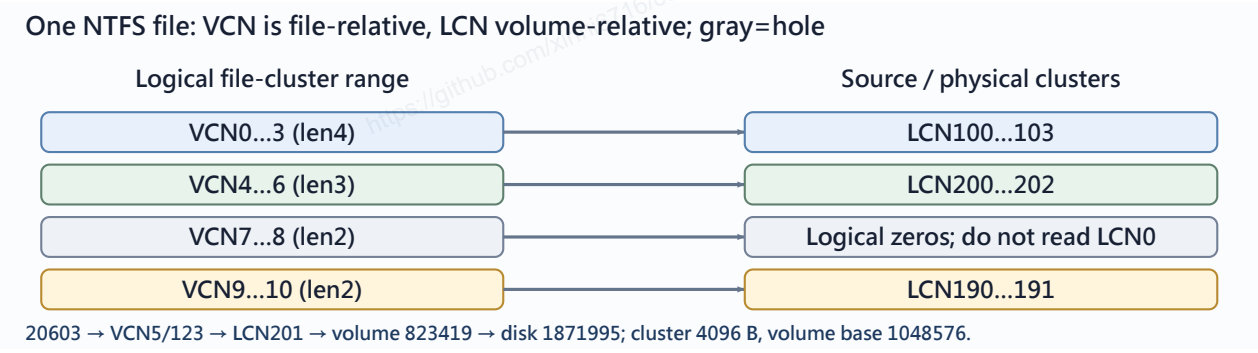
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One complete VCN-to-LCN example

Cluster 4096 B, volume starts at disk byte 1048576. Half-open extents are:

Start VCN	Length	Start LCN	Mapping
0	4	100	VCN0–3→LCN100–103
4	3	200	VCN4–6→LCN200–202
7	2	None	VCN7/8 are holes, logical zeros
9	2	190	VCN9/10→LCN190/191

File byte 20603=5・4096+123 selects extent2. LCN=200+(5−4)=201. Volume byte201・4096+123=823419;disk byte=1048576+823419=1871995. A hole returns zeros without treating None as LCN 0.



File references, indexes, and metadata files

The book's NTFS reference combines 48-bit MFT record and 16-bit sequence. Reuse changes sequence to reject mismatched stale references. Finite 16-bit sequences do not prove everlasting uniqueness against wrap.

Directory indexes sort keys/references. Balanced B+ paths have logarithmic height, but insertion/splitting can reorganize nodes. Small indexes fit root/record;larger ones allocate index blocks. Cached directory sizes/times need not reread current contents on every enumeration.

MFT, bitmap, log, volume, and other metadata use special files and common cache/I/O services. Cluster bitmap, extent, size updates still need coordinated consistency rather than independent successful writes.

Metadata WAL before home updates

This is a single-transaction undo/redo teaching model with exclusive reservation of the new block, not a complete NTFS log encoding or concurrent recovery protocol. All before/after records and a completeness marker must be durable **before any home field is changed**. Complete records and commit are distinguishable and flush is trusted. Steal allows home writes before commit; no-force allows home writes to lag after commit.

Old metadata is size 4096, extents [100], bitmap[200]=0; new metadata is size 8192, extents [100,200], bitmap[200]=1. Persist the complete log before changing home. Announce success only after commit is durable.

Crash prefix	Possible home state	Recovery
Complete log not durable	All old	This model prohibits any home update
Complete log durable, no durable commit	Old or mixed	Undo before-images to complete old
Complete log and commit durable	Old, mixed, or new	Redo after-images to complete new

Recovery chooses the entire metadata state. Actual NTFS also handles LSNs, concurrency and checkpoints.

Recovery invariant and checkpoint scope

In this model, all before/after records and their completeness marker are durable before any home update. Without a durable commit, undo changed fields using before-images; with one, redo all after-images. Assigning old/new values is idempotent, so repeated recovery chooses the same state. Logs must remain available and the flush and failure assumptions must hold.

A checkpoint does not mean every older log can be discarded. Active-transaction undo and dirty-metadata redo remain until safe truncation conditions hold.

Metadata consistency does not make user data an atomic application transaction. Cached write success or NTFS logging does not atomically update two business files. Applications need their own flush, publication, or transaction protocol.

ACLs, encryption, and repair

NTFS uses Windows descriptors/DACLs, not POSIX owner/group/other masks. Traverse-bypass privilege and explicit FILE_TRAVERSE checks are different contexts;Windows does not universally ignore path protection.

EFS encrypts file contents;BitLocker protects volume/storage under its keys/unlock conditions. ACLs alone do not protect unencrypted media read elsewhere. Encryption supplies neither backups nor application transactions.

Bad-sector remapping supplies new locations, not unreadable lost contents without a valid mirror/parity/source. RAID/spares, metadata recovery, and data repair guarantee different things.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sparse allocation, compression, USN, and snapshots

The example spans 11 logical clusters, including two holes. Nine data clusters allocate 36864 B without compression while logical size can be 45056 B. Hole writes later require allocation. This is not a process VA reservation.

Compression units and content/configuration determine compression results, not a fixed ratio. USN tracks changes for consumers/indexes;it differs from recovery logging and can wrap. A history gap requires reconstruction rather than treating it as an eternal backup.

VSS provides a point-in-time storage view. Application updates across related fields can require writer coordination/quiescence for application consistency.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

COW snapshots retain the old version

Live X=41, Y80 creates snapshotS. Before first update X to 42, retain old X=41 for S, then publish new live. After live X=43, S still sees 41 rather than 42. Unchanged Y can share old 80.

Time	Live X	S X	Y
After snapshot	41	41	Shared 80
First write	42	41	80
Second write	43	41	80

Reclaim old backing only after every retaining snapshot/reader releases it. Shared-storage failure can destroy live and snapshot together, so it is not independent backup media.

Interfaces and protocol responsibilities

User Winsock offers sockets;NDIS connects adapter/protocol driver models. TDI is historical;Microsoft directs new Vista-and-later kernel network modules toward WSK. It is not Windows 7's only new-driver route. These responsibilities do not determine a fixed process-switch count per packet.

Mechanism	Role	Does not supply
TCP/IP	Addressing, packets, reliable bytes	Application framing/exactly-once effects
SMB redirector/server	Remote file operations	Identical local/remote failures
Named pipe	Local/remote pipe semantics	Interchangeable byte/message modes
RPC	Request/reply call abstraction	Timeout proves nonexecution
COM	Object interfaces/marshalling	Every call crosses machines

Specify versions, authentication, encryption, errors separately. HTTP/WebDAV/VPN names do not replace settings or imply contemporary security guidance.

UNC paths and partial failures

Opening `\\server\share\report` selects a redirector encoding network requests. Server applies its credentials/filesystem policy and replies status/bytes. Client, network, server, storage can fail independently.

Server writes but reply is lost. Client timeout does not prove no write;retrying append can duplicate. Protocol request identity, deduplication/retry, and durability rules must handle it. TCP reliability alone is not business exactly-once.

Similar APIs can have different share/offline-cache visibility and error timing. Open success does not promise an immortal connection.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Named-pipe messages and directory identities

Fix a message-type named pipe whose receiving handle uses `PIPE_READMODE_MESSAGE`. If the next message has 500 B and the receive buffer holds 200 B, this example can return 200 B with `ERROR_MORE_DATA`; read the remaining 300 B of that same message. FALSE need not mean zero-byte fatal failure. A message-type pipe handle may use byte-read mode, so this example does not describe that mode or TCP framing.

Domains integrate shared identity and management. Active Directory combines directory and authentication-related services rather than merely an encrypted database. Equal display names can have different SIDs; servers check effective credentials, token and object policy.

Offline client caching creates reconnection versions and conflicts. Cached success is not remote durable commit.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Scheduling units and pools

CreateProcess includes an initial thread and separately observable/waitable process/thread handles. Additional threads share address space/resources but have separate PC, stack, TLS. Handle names do not replace references.

Fibers switch in user code within a thread. A blocking syscall on that thread does not let ordinary fiber switching make the same kernel thread run another fiber simultaneously. Windows 7 UMS coordinates user scheduling with managed kernel state under platform/API constraints;it does not make all blocking free or every API fiber-compatible.

Pools reuse workers/queues while retaining work-data lifetimes. One-worker jobA waiting for queued jobB in the same pool can starve/deadlock it.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sharing handles and file positions

Inheritance requires handle inheritability and CreateProcess conditions/handle list. DuplicateHandle requires source/target process authority. Named opening checks the selected namespace/policy again.

DUPLICATE_SAME_ACCESS retains granted mask;copying read-only to a child creates no write right. Different requested access follows object/API rules, not arbitrary amplification or a universal subset theorem.

Duplicated handles to one file object can share position;independent CreateFile can have separate positions. Seek/read races remain. Overlapped operations carry their own offsets/state.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Three ReadFile starting outcomes

Use regular files opened FILE_FLAG_OVERLAPPED, independent OVERLAPPED and valid buffer per outstanding operation, and a manual-reset event. Windows 7 requires non-NULL lpNumberOfBytesRead, so provide a DWORD;pending final bytes come from completion.

Initial result	Meaning	Lifetime
TRUE	Operation completed;inspect actual bytes	Process final result
FALSE, ERROR_IO_PENDING	Submitted, not ordinary failure	Retain buffer, ov, event
FALSE, other error	Follow file/pipe/device contract	Establish whether work remains

Confirm pending work with an appropriate mechanism such as GetOverlappedResult. Do not assume later GetOverlappedResultEx exists in Windows 7.

Six bytes, four bytes, then EOF

Fix a 10-byte regular file ABCDEFGHIJ, with no concurrent write or truncation. Each 6-byte request uses a `FILE_FLAG_OVERLAPPED` handle, its own valid OVERLAPPED and buffer, and an explicit offset. R0 and R1 succeed; initiation may complete immediately or first return pending.

Operation	Offset	Request	Final result	Next step
R0	0	6	Success: 6 B, ABCDEF	Reuse state only after terminal
R1	6	6	Success: 4 B, GHIJ	Completed prefix totals 10 B
R2	10	6	EOF: `ERROR_HANDLE_EOF`	Stop

For this positive-length overlapped regular-file read, R2 may return `ERROR_HANDLE_EOF` at `ReadFile` initiation or after pending through `GetOverlappedResult`; an immediate failure has no pending operation to wait for. A separate non-overlapped synchronous read can instead succeed with zero bytes at EOF. Pipes and devices differ; cancellation can lose to normal completion, so collect the actual result.

Virtual memory, heaps, mappings, and TLS

VirtualAlloc manages reserve/commit/protection. Heap managers subdivide regions;HeapFree need not release the whole region. Sections/views map file/pagefile backing at process-local VAs with compatible offset/size/file bounds.

TLS provides per-thread slots/logical ownership rather than MMU isolation. Another thread with a pointer can access the same process memory. Objects outliving threads need ownership/synchronization.

Commit, working set, and heap counts differ. Reserve 64 KiB, commit 8 KiB, resident 4 KiB can hold simultaneously;one memory number cannot describe all three.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete cancellation and cleanup

While kernel/device may access an operation, retain buffer, OVERLAPPED, event, context. For pendingR, request CancelIoEx(h,&ov), collect terminal through the original completion path, handle normal/aborted/other error and actual bytes, then release data/ov. Close shared handle/event only after all related operations terminate.

Normal completion may win;successful abort can yield ERROR_OPERATION_ABORTED. ERROR_NOT_FOUND from cancellation does not replace original outcome. Never free merely because cancel returned or reuse one ov for simultaneous R1/R2. IOCP/other notification modes have their own delivery rules;follow the selected contract.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

States of one read

Layer	Track	Earlier success can still leave
Authority	Local handle, granted mask, effective token	NoW or later network failure
Virtual memory	Buffer VA, commit, residency, rights	Reserved but uncommitted memory
I/O	IRP/operation, pending, actual bytes	Submission without completion
Cache/filesystem	Offset, VCN/LCN, metadata	Cache write without user durability
Device/network	DMA, connection, storage contract	Requested cancel without stopped hardware

IRQL constrains present kernel activity;thread priority selects ready execution. Confusing them invalidates ISR/DPC waiting and lifetime reasoning.

Reconstructing the examples

The fixed q2 trace gives completions8/9/3 from residual slices/queues. One region can reserve 64 KiB, commit 8, and reside 4 KiB. File byte 20603→VCN5/123→LCN201→volume823419→disk 1871995, with holes separate. Reads at 0/6/10 finish 6/4/EOF; pending, cancel request, terminal differ.

These instantiate general invariants:operations need granted rights;live I/O forbids premature free;extent, size, bitmap agree;one CPU's IRQL does not exclude other CPUs/shared-memory users.

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz et al., Operating System Concepts, 9th ed., Ch. 19. Microsoft: IRQL, VirtualAlloc, AccessCheck. ReadFile, EOF, CancelIoEx, named pipes.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study